

FAST-NUCES ISLAMABAD

ARTIFICIAL INTELLIGENCE

ASSIGNMENT # 1

Pacman Search Game

Submitted by

Fahima Sohail Ahmed (24i-3116)
Wania Bakhat (24i-3040)

Submitted to: Dr. Muhammad Bilal
FAST-NUCES Islamabad

Course: Artificial Intelligence (AI2002)

1. Introduction and Objectives

This project implements classical search techniques in the Pacman Search Game. The work covers DFS, BFS, UCS, Greedy Best-First Search (GBFS), A* Search, multi-goal corner search, food search, closest-dot search, CSV trace logging, and an original custom maze.

The assignment requires the main search algorithms in `search.py`, state-space formulations and heuristics in `searchAgents.py`, a custom layout, execution evidence, CSV traces, and a 6–10 page report.

- Implement graph-search DFS and BFS with the provided Stack and Queue.
- Implement UCS with accumulated path cost $g(n)$ and frontier priority updates.
- Implement GBFS with heuristic-only priority $h(n)$ and command-line heuristic selection.
- Implement A* with $f(n)=g(n)+h(n)$.
- Represent CornersProblem with position plus visited-corner information.
- Implement food and closest-dot search and heuristic support.
- Generate step-by-step CSV traces containing the required fields.
- Evaluate all five algorithms on the original `i243116Search` custom layout.

2. Algorithm Implementations

2.1 DFS and BFS

DFS uses `util.Stack` and an explicit explored set, producing LIFO graph search. BFS uses `util.Queue` and marks states when first enqueued, giving shortest path length when every action has unit cost.

2.2 UCS

UCS uses `util.PriorityQueue` ordered by $g(n)$. The implementation stores the best known cost and calls `PriorityQueue.update` when a cheaper path is found.

2.3 GBFS and A*

GBFS prioritizes $h(n)$ only. A* prioritizes $f(n)=g(n)+h(n)$, combining the cost already spent with a heuristic estimate. With an admissible and consistent heuristic, A* can return an optimal path.

3. State Spaces and Heuristics

3.1 Position Search

A position-search state is Pacman's (x,y) location. Legal neighboring positions are generated with their action and step cost.

3.2 CornersProblem

The state is (position, visitedCorners), where `visitedCorners` is a four-element Boolean tuple. The goal is reached when all four corners have been visited. Each legal move has cost 1.

- Initial state records whether Pacman starts on a corner.
- Successors create a new tuple whenever a corner is reached.
- The goal test checks that every corner flag is True.

3.3 Corners Heuristic

The submitted heuristic repeatedly connects the current position to the nearest unvisited corner using Manhattan distance. This is useful because Manhattan distance is a lower bound on wall-aware distance. However, a formal admissibility/consistency proof for the particular greedy nearest-corner chaining strategy is not included in the supplied project; the report therefore avoids claiming a proof that is not demonstrated.

3.4 FoodSearchProblem

`FoodSearchProblem` represents each state as (position, foodGrid). The goal is an empty food grid. The submitted `foodHeuristic` uses the maximum maze distance from Pacman's current position to any remaining food dot and caches computed distances in `heuristicInfo`.

3.5 ClosestDotSearchAgent

AnyFoodSearchProblem treats any food-containing cell as a goal. ClosestDotSearchAgent repeatedly runs BFS to reach a closest available dot, appends that path segment, and continues until no food remains.

4. CSV Trace Logging and Complexity

search.py contains a common logger that writes the required columns: iteration, expanded_state, parent, action, generated_successors, frontier_before, frontier_after, explored, g, h, and f. The supplied evidence contains one trace file for each of DFS, BFS, UCS, GBFS, and A*.

Algorithm	Rows	Final g	Final h	Final f	Final state
DFS	53	52	0	52	(1, 1)
BFS	89	40	0	40	(1, 1)
UCS	89	40	0	40	(1, 1)
GBFS	41	40	0	40	(1, 1)
ASTAR	76	40	0	40	(1, 1)

The values above are read directly from the final row of each supplied CSV. The included traces all end at state (1,1).

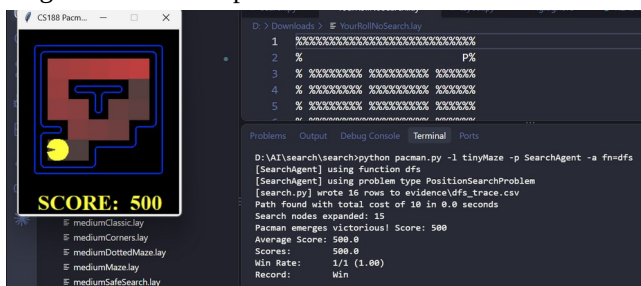
4.1 Complexity Summary

Algorithm	Time (standard worst case)	Space (standard worst case)	Priority
DFS	$O(b^m)$	$O(bm)$	LIFO
BFS	$O(b^d)$	$O(b^d)$	FIFO
UCS	Exponential in solution cost	Exponential in solution cost	$g(n)$
GBFS	$O(b^m)$ worst case	$O(b^m)$ worst case	$h(n)$
A*	Exponential worst case	Exponential worst case	$g(n)+h(n)$

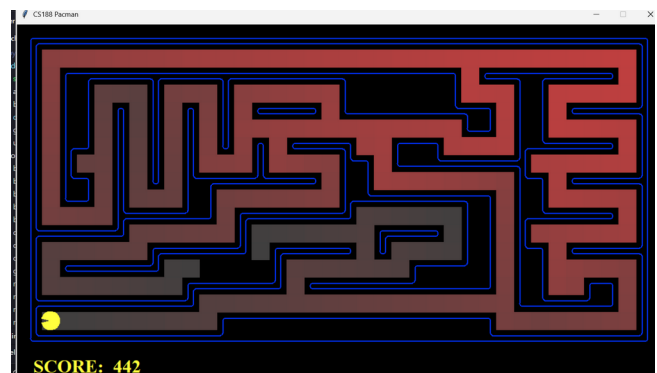
For multi-goal search, the state space grows because the state stores additional goal information. Corners has a position plus four Boolean values; FoodSearch additionally stores the remaining-food grid. This makes heuristic quality increasingly important.

5. Standard-Maze Experimental Evidence

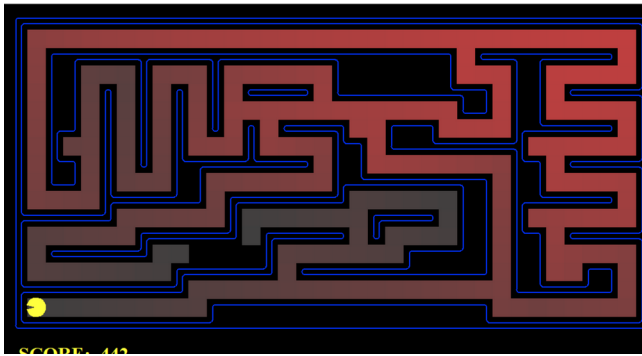
The following figures are the original screenshots included in the submitted evidence folder. They are reproduced without altering the Pacman output.



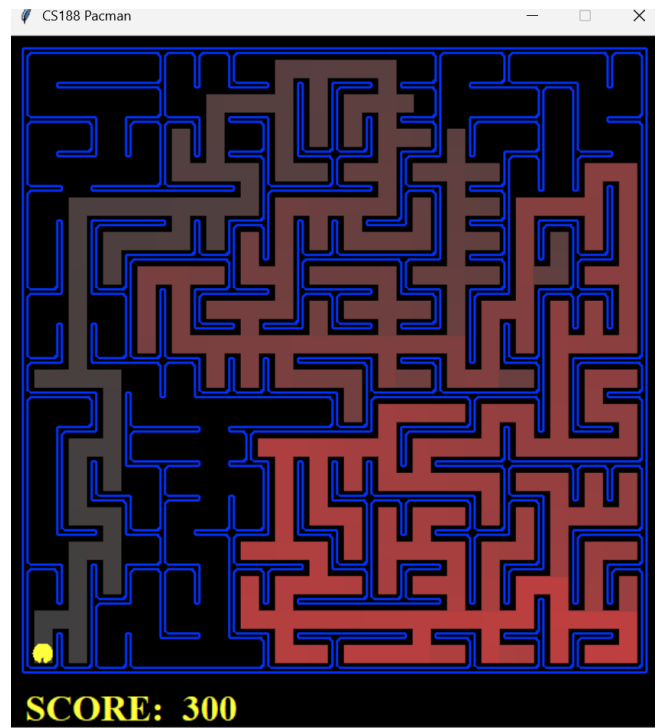
DFS — tinyMaze



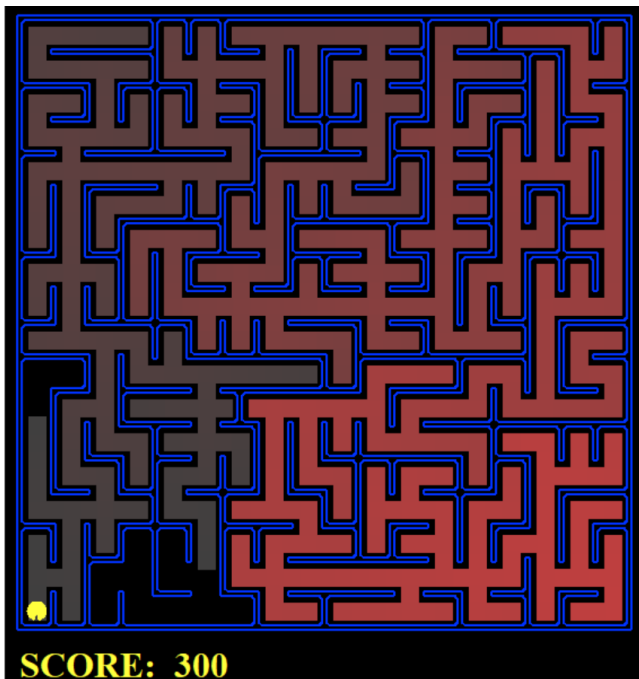
BFS — mediumMaze



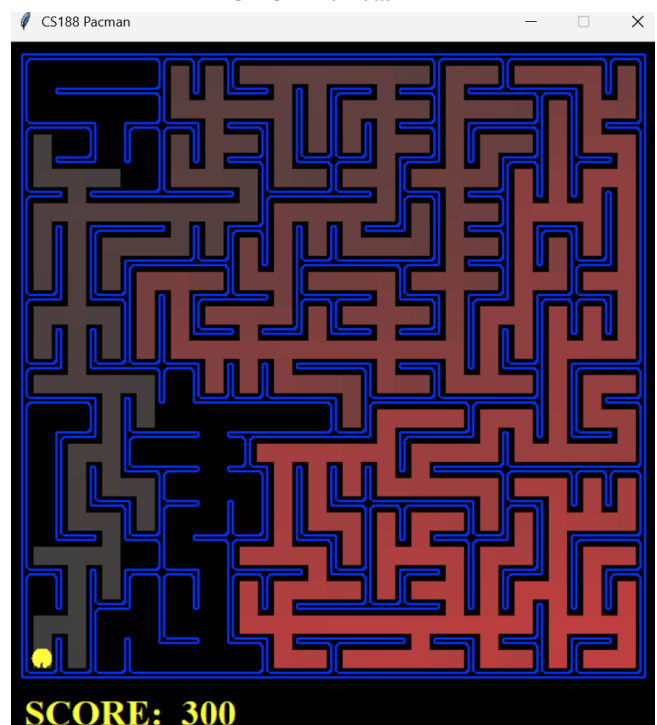
UCS — mediumMaze



GBFS — Manhattan



A* — null heuristic



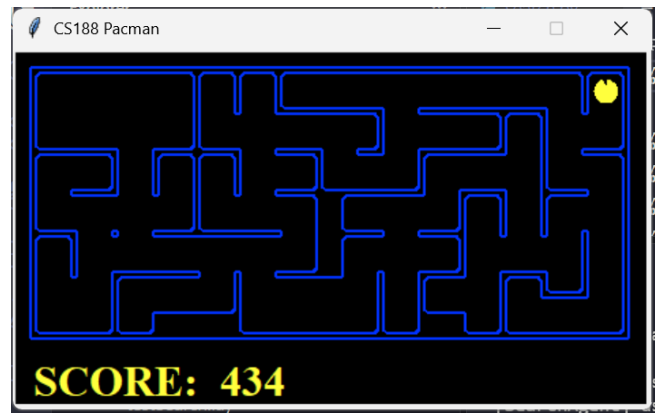
A* — Manhattan

The screenshots show successful completion of the displayed searches. The A* pair also demonstrates the difference between $h(n)=0$ and a Manhattan-distance heuristic.

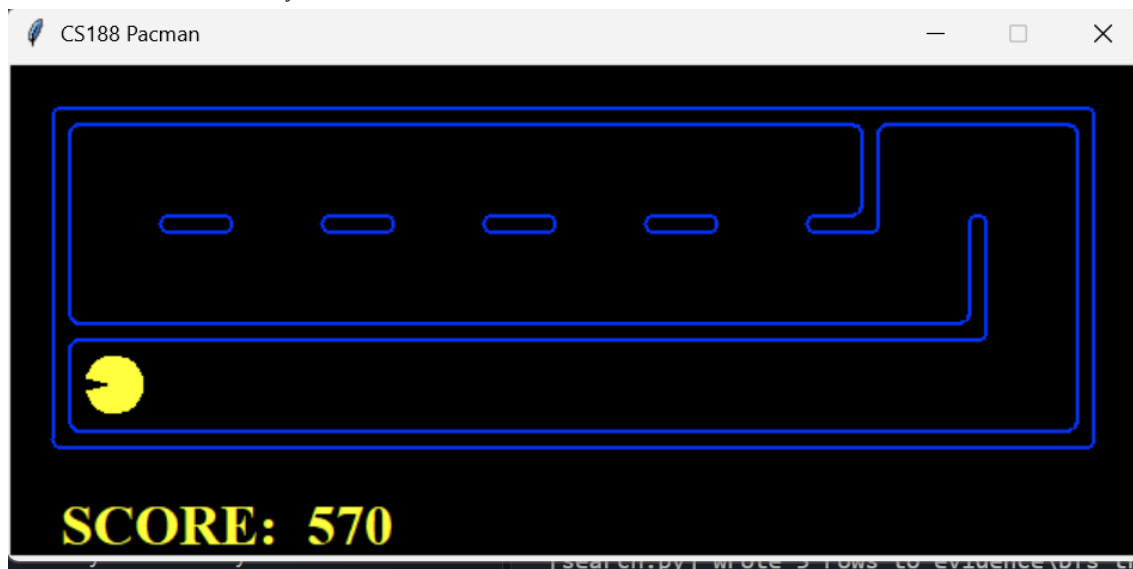
6. Multi-Goal and Food-Search Evidence



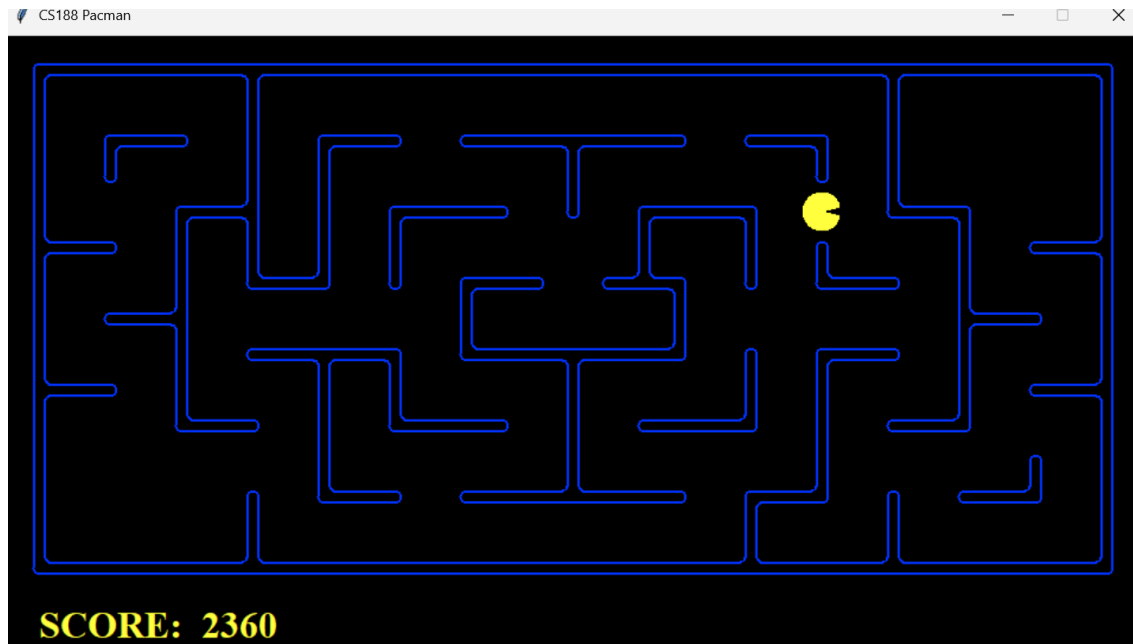
CornersProblem — tinyCorners



A Corners — mediumCorners*



A Food Search on trickySearch — supplied execution screenshot.*

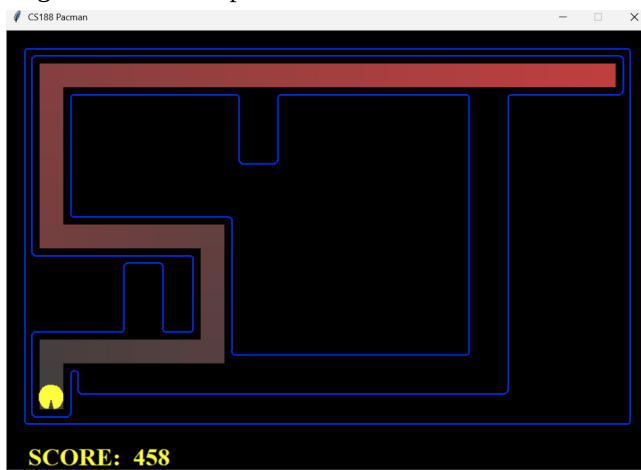


Closest-dot search on bigSearch — supplied execution screenshot.

These figures correspond to the state-space extensions implemented in `searchAgents.py`: tracking visited corners, tracking remaining food, and repeatedly solving the any-food problem with BFS.

7. Custom Maze Experiment

The custom layout is `layouts/i243116Search.lay`. It is a 27×17 maze containing long corridors, branching choices, dead-end regions, and a deceptive route structure suitable for comparing search behavior.



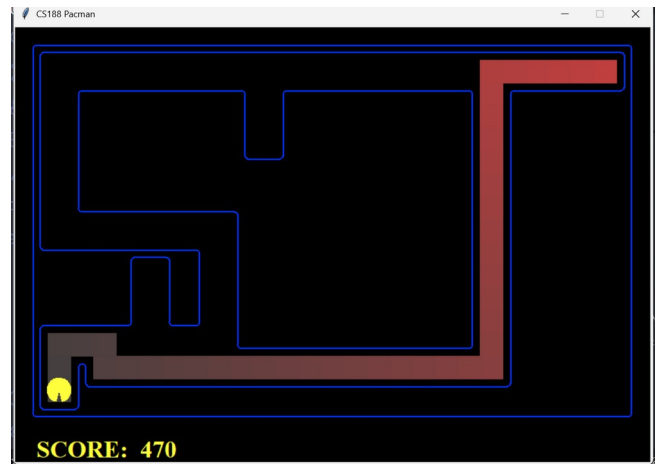
DFS



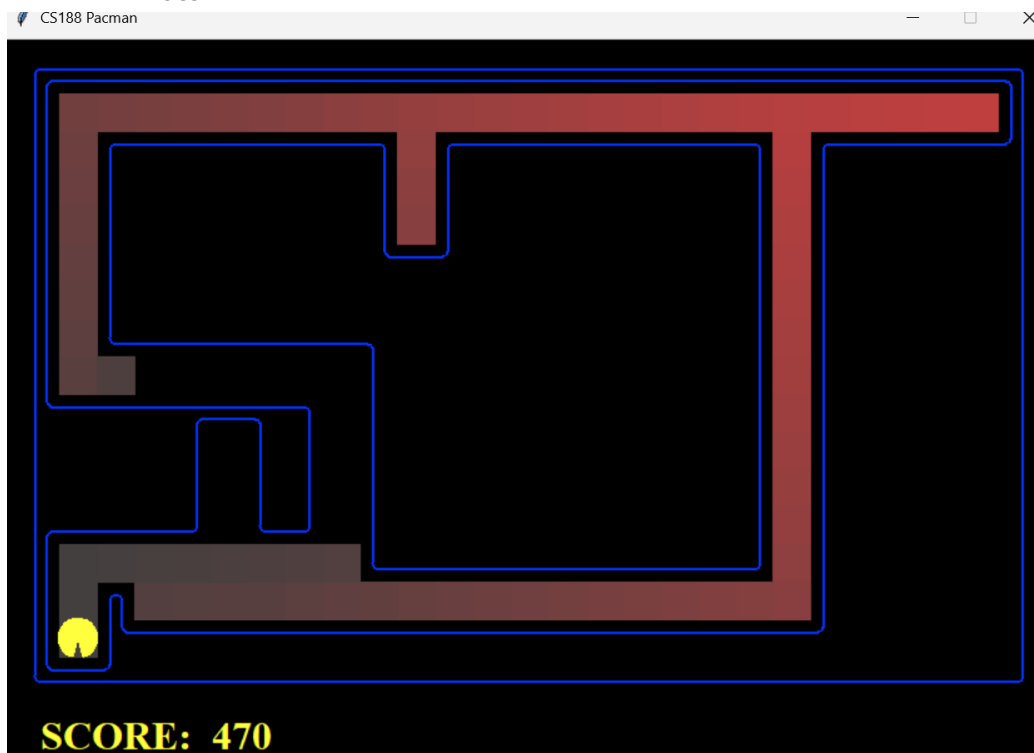
BFS



UCS



GBFS — Manhattan



A* — Manhattan heuristic on the same custom layout.

7.1 Observed Comparison

The supplied screenshots show that all five algorithms can solve the custom maze, while the displayed paths/search behavior differ. The custom DFS screenshot has a different displayed score from the other four custom screenshots. Because no separate custom-layout expansion table is supplied, numerical expansion counts are not invented here.

8. Results Discussion

The supplied CSV traces provide a concrete numerical comparison for their recorded runs. DFS expands 53 states and reaches the goal with $g=52$. BFS and UCS each expand 89 states and reach the goal with $g=40$. GBFS expands 41 states and reaches the goal with $g=40$. A* expands 76 states and reaches the goal with $g=40$.

These values illustrate the distinction between solution cost and search effort. DFS can reach a goal without minimizing path length. BFS is optimal in unit-cost mazes. UCS accounts for path cost explicitly. GBFS uses only heuristic direction and may reach a goal quickly without guaranteeing a minimum-cost path. A* combines $g(n)$ and $h(n)$, using both accumulated cost and estimated remaining cost.

8.1 Evidence Limitations

The supplied evidence folder contains five algorithm-level CSV files. The shared A* logger writes to `astar_trace.csv`, so a separate permanent CSV is not present for every individual multi-goal A* invocation. Likewise, the custom-maze screenshots provide visual evidence but not a separate numeric expansion table. These are limitations of the supplied evidence package, not values that should be fabricated in the report.

9. Conclusion

The project demonstrates the requested search algorithms, multi-goal state spaces, heuristic-driven search, CSV tracing, and custom-maze experimentation. The implementation uses the provided search data structures and produces both numerical trace evidence and graphical Pacman evidence. The included README and report document the commands, project structure, algorithms, state representations, heuristic ideas, complexity, and observed results.

10. Submission Checklist

- ✓ `search.py` — DFS, BFS, UCS, GBFS, A*, and trace logging.
- ✓ `searchAgents.py` — CornersProblem, corner heuristic, FoodSearchProblem, food heuristic, and closest-dot search.
- ✓ `layouts/i243116Search.lay` — original custom layout.
- ✓ `evidence/*.csv` — supplied algorithm traces.
- ✓ `evidence/screenshots/` — supplied graphical evidence.
- ✓ `README.txt` — setup and run commands.
- ✓ `report.pdf` — final report.